NOVA SCHOOL OF SCIENCE & TECHNOLOGY

STR

LAB #1 – class 4/5

freeRTOS: TASKS & SYNCHRONIZATION WITH SEMAPHORES & COMMUNICATION WITH MAILBOXES

2025 - 2026

OBJECTIVES

freeRTOS: Tasks and Synchronization with Semaphores & communication with mailboxes

- ❑ Task Creation Review
- ❑ Race condition Review
- ❑ Semaphores Review
- ❑ Mailboxes
- ❑ Labwork 1: First tasks identification and interactions (synchronization & communication)
- ❑ Interrupts Review
- ❑ Task Control (task suspend & task resume)

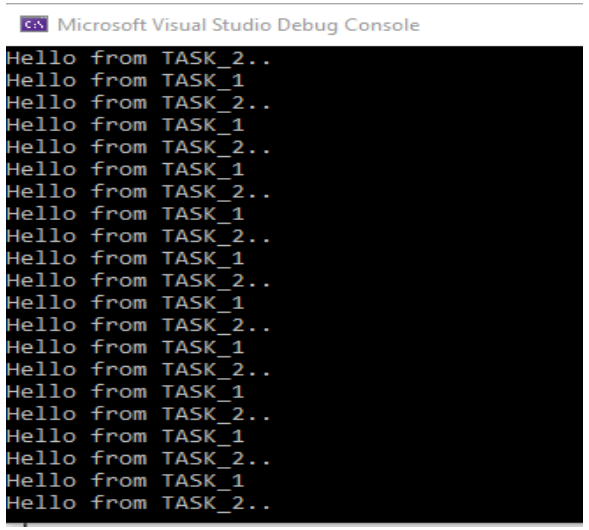
TASK CREATION: Parallel tasks running INDEPENDENTLY

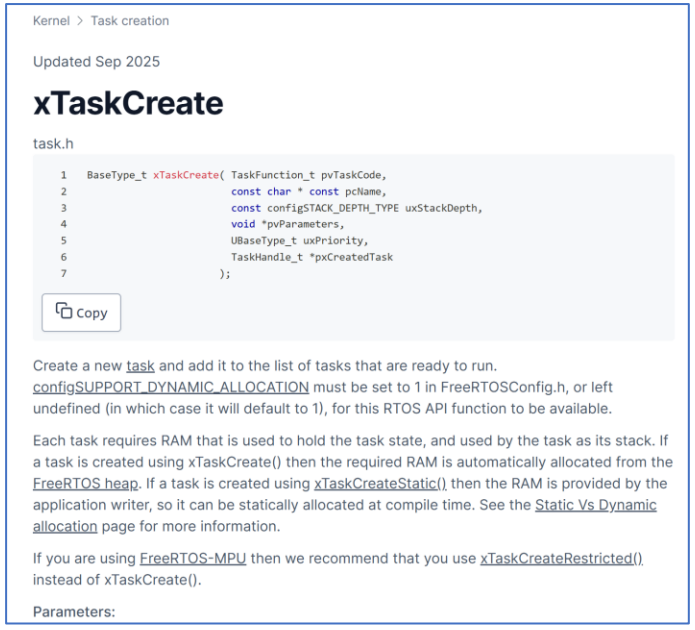
REVIEW

```
void myDaemonTaskStartupHook(void) {
     // runs only once;
     // used for code initialization
     xTaskCreate(vTaskCode_2, "Task2", 100, NULL, 0, NULL);
     xTaskCreate(vTaskCode_1, "Task1", 100, NULL, 0, NULL);
}
```

```
int main()
{
    initialiseHeap();
    vApplicationDaemonTaskStartupHook = &myDaemonTaskStartupHook;
    vTaskStartScheduler();
}
```

```
xTaskCreate(
     pvTaskCode,      // pointer to function with task code (void *)
     pcName,          // task Name  (char *)
     usStackDepth,    // stack size for the task
     pvParameters,    // initial array parameters for the task
     uxPriority,      // task priority (higher value means higher priority
     pxCreatedTask    // handle to task (can be null)
)
```





Kernel > Task creation

Updated Sep 2025

xTaskCreate

task.h

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,
                        const char * const pcName,
                        const configSTACK_DEPTH_TYPE uxStackDepth,
                        void *pvParameters,
                        UBaseType_t uxPriority,
                        TaskHandle_t *pxCreatedTask
                      );
```

Copy

Create a new task and add it to the list of tasks that are ready to run. configSUPPORT_DYNAMIC_ALLOCATION must be set to 1 in FreeRTOSConfig.h, or left undefined (in which case it will default to 1), for this RTOS API function to be available.

Each task requires RAM that is used to hold the task state, and used by the task as its stack. If a task is created using xTaskCreate() then the required RAM is automatically allocated from the FreeRTOS heap. If a task is created using xTaskCreateStatic() then the RAM is provided by the application writer, so it can be statically allocated at compile time. See the Static Vs Dynamic allocation page for more information.

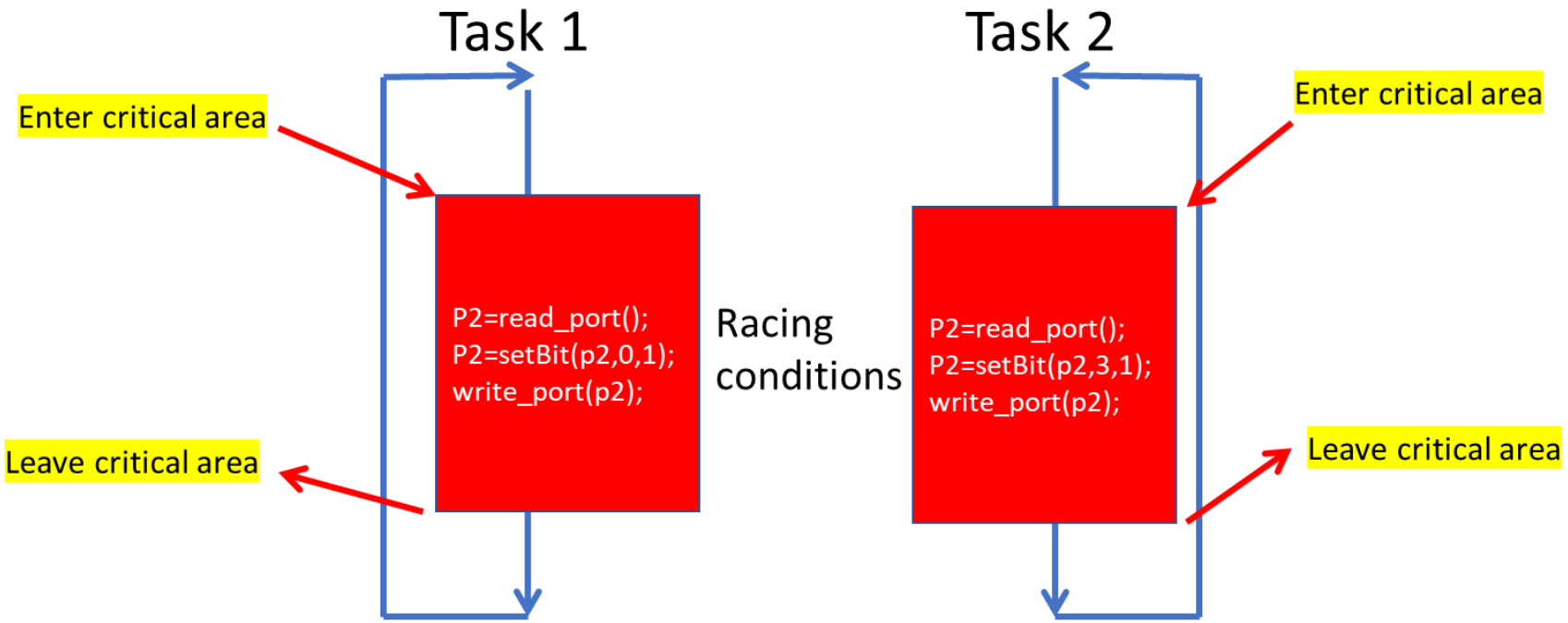
If you are using FreeRTOS-MPU then we recommend that you use xTaskCreateRestricted() instead of xTaskCreate().

Parameters:

Race Condition: Splitter Conveyor Kit (access to Port 2)

REVIEW

- Task 1 and Task 2 are running a fragment of code, which cannot be executed simultaneously
- We need to specify critical areas / sections.
- A critical area is a block of code which while running, everything else in the system freezes until the code inside the block finishes and execution exits from that area.



Race Condition: Splitter Conveyor Kit (access to Port 2)

- **taskENTER_CRITICAL()** and **taskEXIT_CRITICAL()** operate at kernel level, meaning when inside a critical area, “everything” else inside the kernel stops working until exiting the critical area.
- As such, the code inside critical areas should work as fast as possible (no loops, no sleeps,… no anything causing latency).
- For less strident conditions, the recommended alternative is to model mutual exclusion with semaphores.

REVIEW

```
void vTaskCylinder1(void* pvParameters){
                uInt8 aa, sensor;
                 while (TRUE){
                        //go front
                        taskENTER_CRITICAL();
                         uInt8 aa = readDigitalU8(2); setBitValue(&aa, 3, 0); setBitValue(&aa, 4, 1);
                         vTaskDelay(10); // to simulate some latency
                         writeDigitalU8(2, aa);
                         taskEXIT_CRITICAL();
                          // wait until front sensor
                          sensor = readDigitalU8(0);
                          while (getBitValue(sensor,3)){
                                sensor = readDigitalU8(0); vTaskDelay(1);
                           }
                          // go back
                           taskENTER_CRITICAL();
                           aa = readDigitalU8(2); setBitValue(&aa, 3, 1);  setBitValue(&aa, 4, 0);
                            vTaskDelay(10); // to simulate some latency
                             writeDigitalU8(2, aa);
                             taskEXIT_CRITICAL();
                 // wait until back sensor
                 sensor = readDigitalU8(0);
                            while (getBitValue(sensor,4)){
                        sensor = readDigitalU8(0); vTaskDelay(1);
                  }
                }
}
```

Example of a CRITICAL AREA - Splitter Conveyor (access to Port 2)

- If both task_1 and task_2 run slightly at the same time, they might read the same value from port 2.
- In this situation, the value for port 2 may overlap.
- As such, either the system will run cylinder 1 front, cylinder 2 back, or both. But we don't know which.
- We need to ensure that one task works without undermining other tasks.

```
void task_1(void *)
{
     moveCylinder1Front();
}

void moveCylinder1Front()
{
     uInt8 p;
     p = readDigitalU8 (2);
     setBitValue(&p,4,1);
     setBitValue(&p,3,0);
     writeDigitalU8 (2,p);
}
```

```
void task_2(void *)
{
      moveCylinder2Back();
}

void moveCylinder2Back()
{
     uInt8 p;
     p = readDigitalU8 (2);
     setBitValue(&p,5,1);
     setBitValue(&p,6,0);
     writeDigitalU8 (2,p);
}
```

Example of a CRITICAL AREA - Splitter Conveyor (access to Port 2)

Solution: Set code as **critical**, so that it can run in exclusive mode. This means no other tasks or interrupts can run before exiting the critical code.

But be careful: if loops or long operations are created inside **critical** areas, the whole program, including the kernel, will freeze until exiting that critical area (see next slide).

```
void moveCylinder1Front()
{
	uInt8 p;
	taskENTER_CRITICAL();
	p = readDigitalU8 (2);
	setBitValue(&p,4,1);
	setBitValue(&p,3,0);
	writeDigitalU8 (2,p);
	taskEXIT_CRITICAL();
}
```

```
void moveCylinder2Back()
{
	uInt8 p;
	taskENTER_CRITICAL();
	p = readDigitalU8 (2);
	setBitValue(&p,5,1);
	setBitValue(&p,6,0);
	writeDigitalU8 (2,p);
	taskEXIT_CRITICAL();
}
```

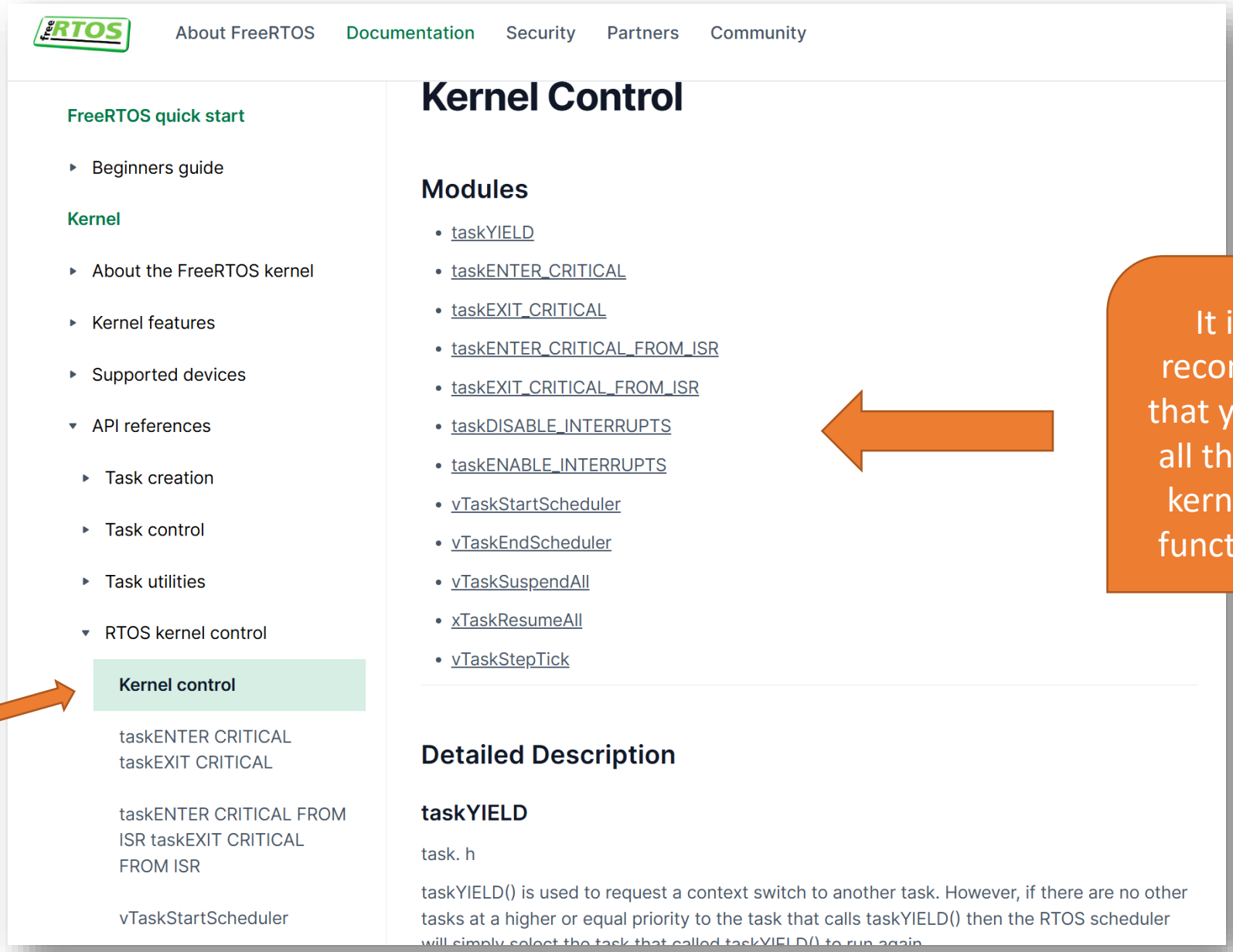
Example of a CRITICAL AREA - Splitter Conveyor (access to Port 2)

```
void some_silly_function()
{
    taskENTER_CRITICAL();
     while(some_lasting_condition) {

do_something_taking_long_time();
    }
    taskEXIT_CRITICAL();
}
```

All the other TASKS and interrupts will freeze during the while loop, basically, "killing" the system.

Therefore, critical areas should contain very fast code ONLY !!!!

freeRTOS API – Kernel Control



www.freertos.org

It is highly recommended that you explore all the possible kernel control functionalities!

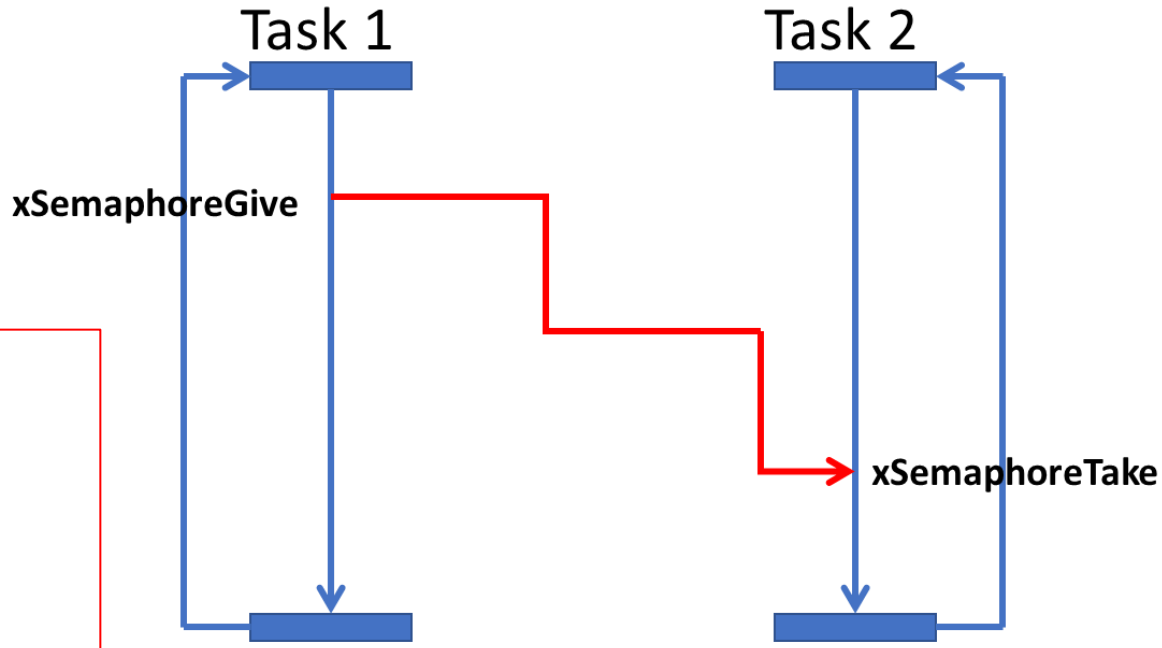
TASK CREATION: Parallel tasks with SYNCHRONIZATION

REVIEW

```
// declare the semaphore on top of the file, just below #defines

xSemaphoreHandle xSemaphore = NULL;
```

```
void vTask_signals(void* pvParameters)
{
    printf("write something\n");
    while (1) {
        char ch[101];
        gets_s(ch, 100);  // input from keyboard
        printf("\n vTask_signals got string '%s' from keyboard…", ch);
        if (strcmp(ch, "fim") == 0)
            exit(0); // terminate program...
        xSemaphoreGive(xSemaphore); //V
    }
}
```

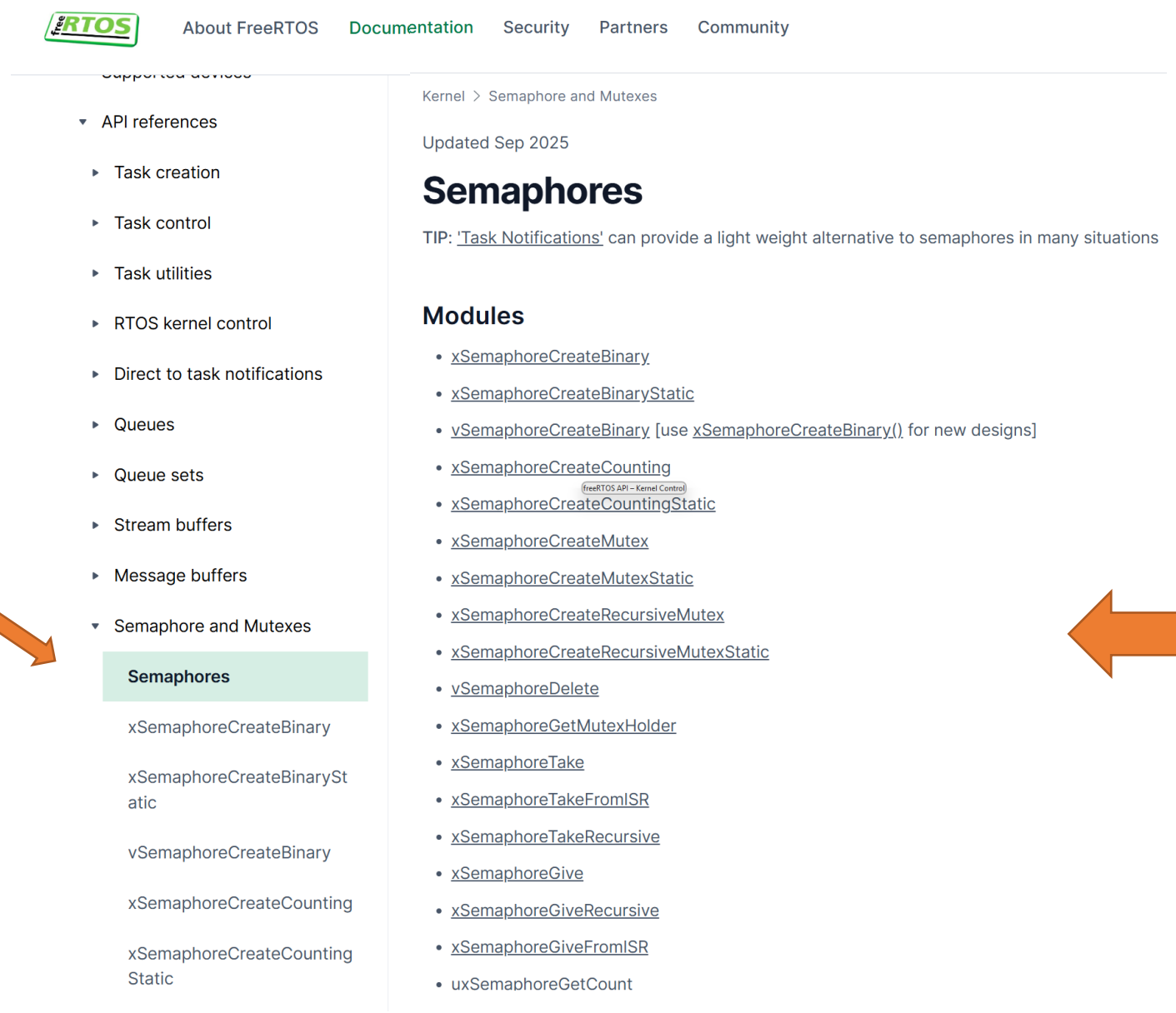
```
void vTask_waits(void* pvParameters)
{
    while (1) {
        if (xSemaphoreTake(xSemaphore, 10000) == pdTRUE) //P
            printf("\nvTask_waits has been awaken\n");
        else
            printf("\nvTask_waits is tired of waiting\n");
    }
}
```




```
void myDaemonTaskStartupHook(void) {
    // runs only once;
    // used for code initialization

    xSemaphore = xSemaphoreCreateCounting(10, 0);
    xTaskCreate(vTask_waits, “vTask_waits", 100, NULL, 0, NULL);
    xTaskCreate(vTask_signals, “vTaskCode2", 100, NULL, 0, NULL);
}
```

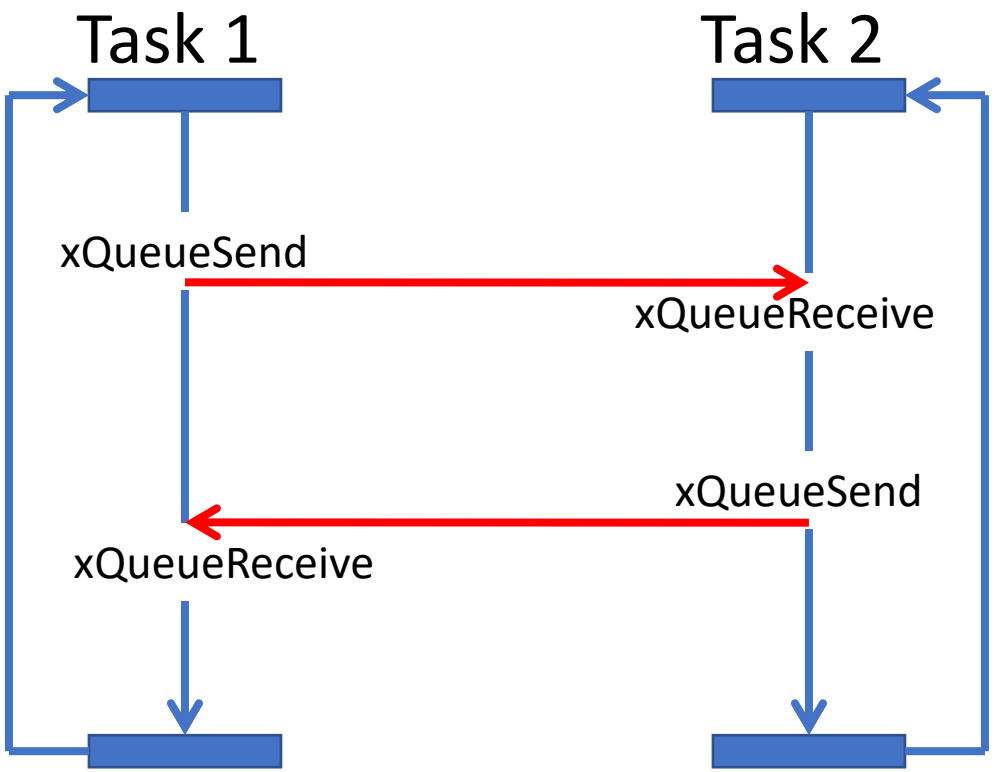
```
int main()
{
    initialiseHeap();
    vApplicationDaemonTaskStartupHook = &myDaemonTaskStartupHook;
    vTaskStartScheduler();
}
```

freeRTOS API – Semaphores / Mutexes



It is highly recommended that you explore all semaphore typology and functionalities!

TASK COMMUNICATION (MAILBOXES / QUEUES)



- Two parallel tasks sending/ receiving data through MAILBOXES
- One task sends a message to another and vice-versa

TASK COMMUNICATION (MAILBOXES / QUEUES)

- Let's update project **demo_rtos.sln** (in C:\str\demo_rtos) with mailboxes

```
// declare the mailbox on top of the file, just below #defines
xQueueHandle xQueue = NULL;
```

```
void vTaskCode_MbxSender (void * pvParameters){
double val = 0;
   for (;; ){
      // send real numbers to another task
      // sender is faster than receiver
      val = val + 1;
      xQueueSend(xQueue, &val, 0);
      vTaskDelay(50); //50ms
   }
}
```

```
void vTaskCode_MbxReceiver (void * pvParameters){
   double val;
   for (;; ){
      // receive real numbers from the sender
      // sender is faster than receiver
      xQueueReceive(xQueue, &val, portMAX_DELAY);
      printf("\nreceived_val =%1.3f", val);
      vTaskDelay(1000); //1s
   }
}
```

```
void myDaemonTaskStartupHook (void) {
   // declare and create a mailbox
   xQueue = xQueueCreate(10, sizeof(double));

   xTaskCreate(vTaskCode_MbxReceiver, "vTaskCode_MbxReceiver ", 100, xQueue, 0, NULL);
   xTaskCreate(vTaskCode_MbxSender, "vTaskCode_MbxSender ", 100, xQueue, 0, NULL);
   (...)
}
```

```
void main(){
     initialiseHeap();
     vApplicationDaemonTaskStartupHook = &myDaemonTaskStartupHook;
     vTaskStartScheduler();
}
```

What happens?

TASK COMMUNICATION (MAILBOXES / QUEUES)

C:\str\demo_rtos\Debug\demo_rtos.

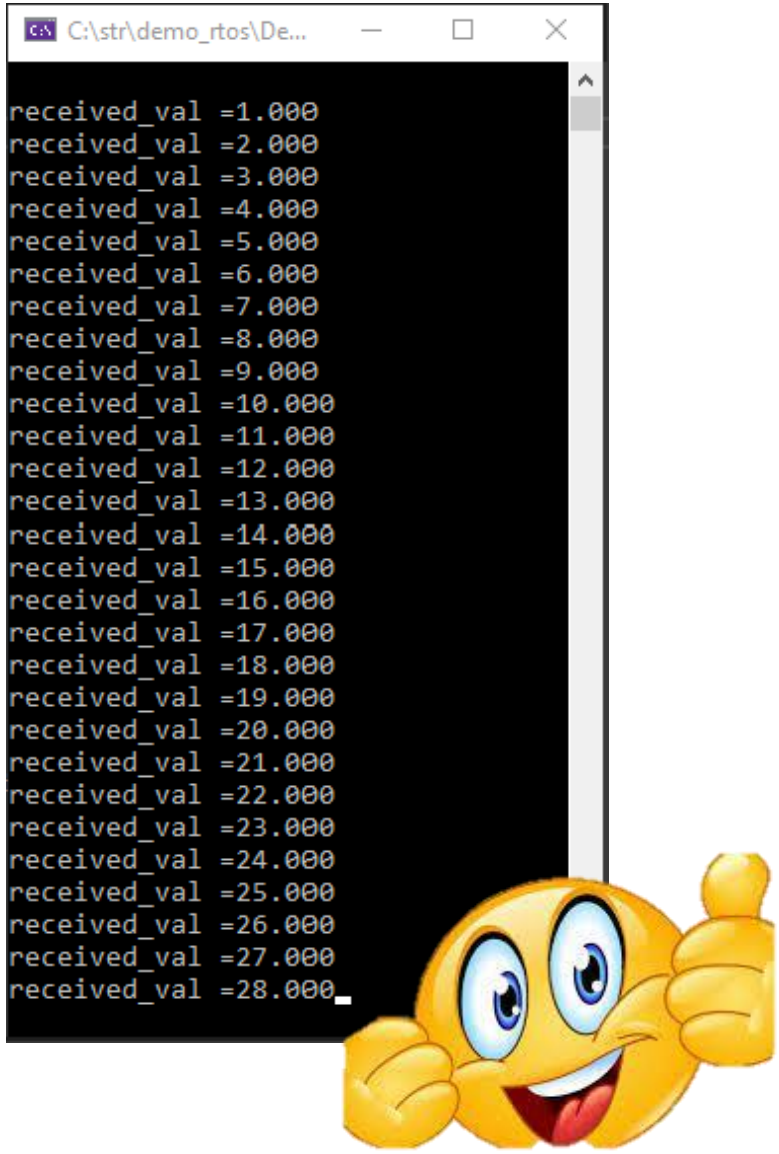
```
received_val =1.000
received_val =2.000
received_val =3.000
received_val =4.000
received_val =5.000
received_val =6.000
received_val =7.000
received_val =8.000
received_val =9.000
received_val =10.000
received_val =11.000
received_val =21.000
received_val =41.000
received_val =61.000
received_val =82.000
received_val =103.000
received_val =124.000
received_val =144.000
received_val =183.000
received_val =203.000
received_val =223.000
received_val =243.000
received_val =263.000
received_val =283.000
received_val =303.000
received_val =323.000
received_val =343.000
```

- Now try to change the *vTaskDelay* to **1s** in *vTaskCode_MbxSender:*

```
void vTaskCode_MbxSender (void * pvParameters){
   xQueueHandle xQueue = (xQueueHandle)pvParameters;
   double val = 0;
   for (;; ){
      // send real numbers to another task
      // sender is faster than receiver
      val = val + 1;
      xQueueSend(xQueue, &val, 0);
      vTaskDelay(1000);
   }
}
```

And now?
What happens?

TASK COMMUNICATION (MAILBOXES / QUEUES)



- Now try to change the *Mbx* size to 100 and keep the *vTaskDelay* to **50ms** in *vTaskCode_MbxSender:*

```
void myDaemonTaskStartupHook (void) {
   // declare and create a mailbox
   xQueueHandle xQueue = xQueueCreate(100, sizeof(double));

   xTaskCreate(vTaskCode_MbxReceiver, "vTaskCode_MbxReceiver ", 100, xQueue, 0, NULL);
   xTaskCreate(vTaskCode_MbxSender, "vTaskCode_MbxSender ", 100, xQueue, 0, NULL);
   (...)
}
```

What is your conclusion?

TASK COMMUNICATION (MAILBOXES / QUEUES)

- Let's send a structured message with Part information (structure)

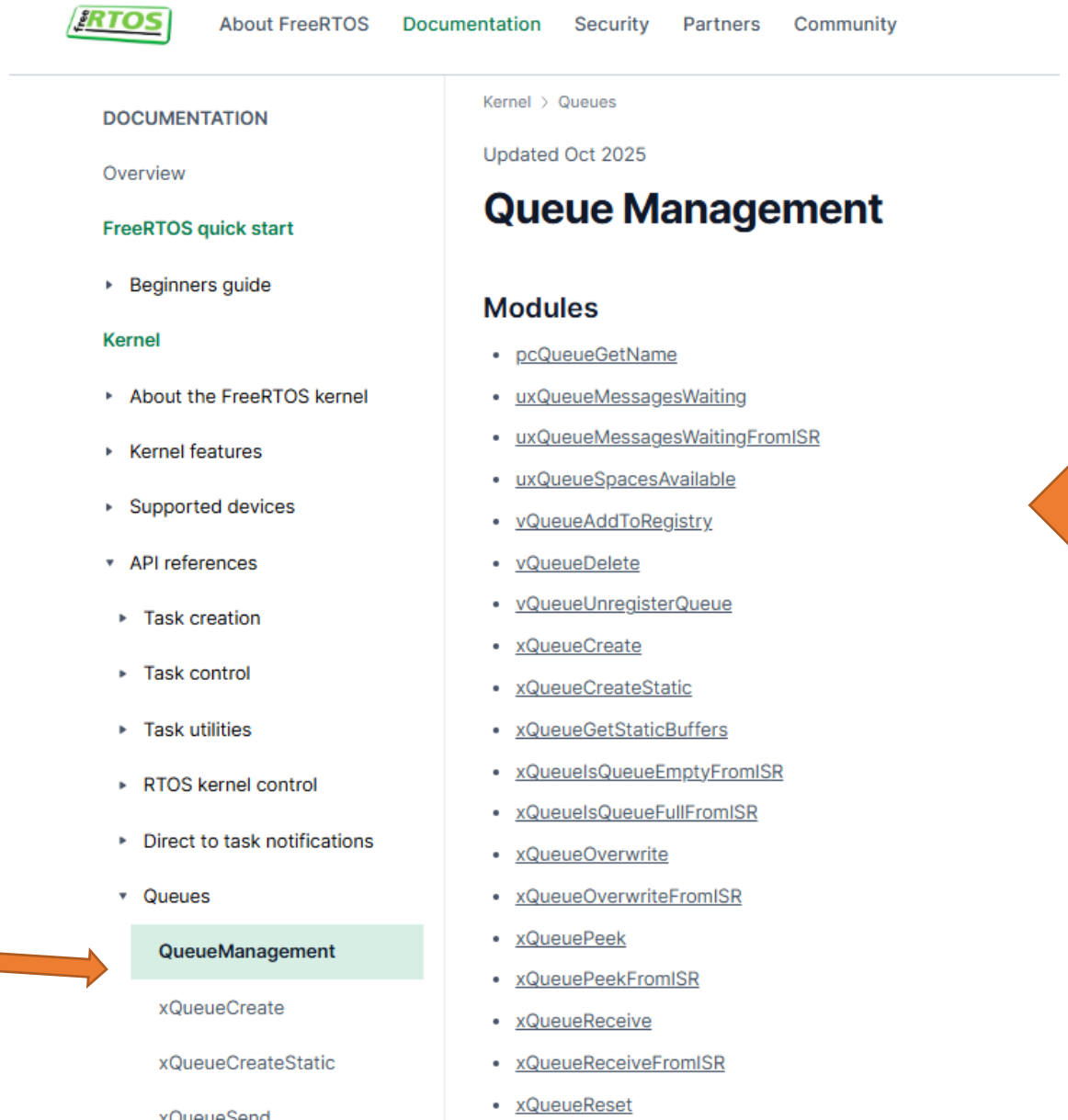
```
// declare the mailbox on top of the file, just below #defines
xQueueHandle mbx_part;

typedef struct part {
   char package_ref[50];
   int  destination;
} Part;
```

```
void myDaemonTaskStartupHook(void) {
   mbx_part = xQueueCreate(10, sizeof(Part));
   xTaskCreate(vTaskCode_MbxReceiver, "vTaskCode_MbxReceiver ", 100, NULL, 0, NULL);
   xTaskCreate(vTaskCode_MbxSender, "vTaskCode_MbxSender ", 100, NULL, 0, NULL);
}
```

```
void vTaskCode_MbxSender(void* pvParameters){
   Part part_info;

   strcpy_s(part_info.package_ref, sizeof(part_info.package_ref), "123ABC");
   part_info.destination = 2;

   xQueueSend(mbx_part, &part_info, portMAX_DELAY);
}
```

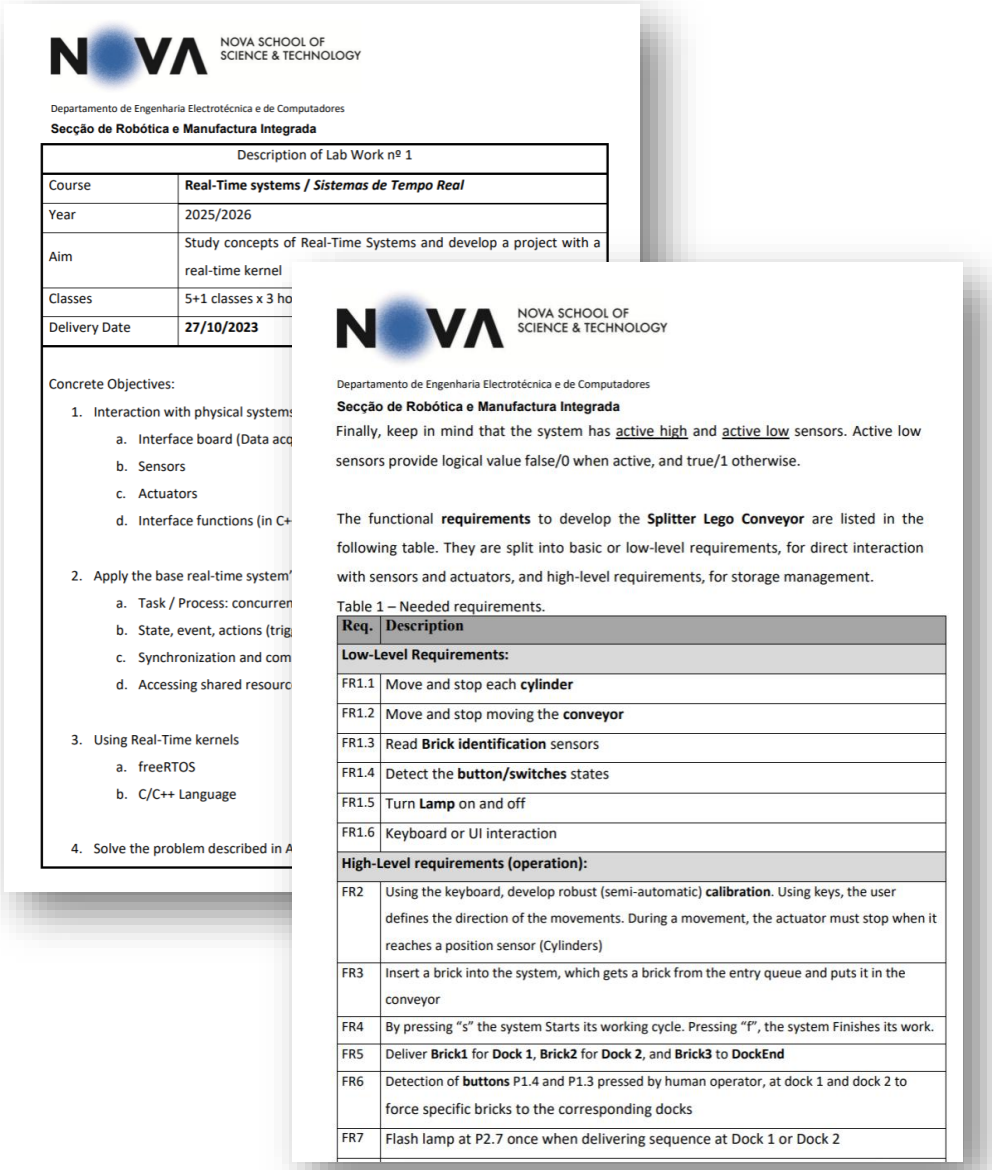
```
void vTaskCode_MbxReceiver(void* pvParameters){
   Part part_info;

   xQueueReceive(mbx_part, &part_info, portMAX_DELAY);
   printf("\n PlaPackage Ref = %s\n Destination = %d\n",
   part_info.package_ref, part_info.destination);
}
```

freeRTOS API – Mailboxes/Queues



It is highly recommended that you explore all mailboxes/queues typology and functionalities!

Lab Work 1



NOVA SCHOOL OF SCIENCE & TECHNOLOGY

Departamento de Engenharia Electrotécnica e de Computadores

Secção de Robótica e Manufactura Integrada

Description of Lab Work nº 1	
Course	**Real-Time systems / *Sistemas de Tempo Real***
Year	2025/2026
Aim	Study concepts of Real-Time Systems and develop a project with a real-time kernel
Classes	5+1 classes x 3 ho
Delivery Date	**27/10/2023**

Concrete Objectives:

1. Interaction with physical systems
   a. Interface board (Data acq
   b. Sensors
   c. Actuators
   d. Interface functions (in C+
2. Apply the base real-time system'
   a. Task / Process: concurren
   b. State, event, actions (trig
   c. Synchronization and com
   d. Accessing shared resourc
3. Using Real-Time kernels
   a. freeRTOS
   b. C/C++ Language
4. Solve the problem described in A

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

Departamento de Engenharia Electrotécnica e de Computadores

Secção de Robótica e Manufactura Integrada

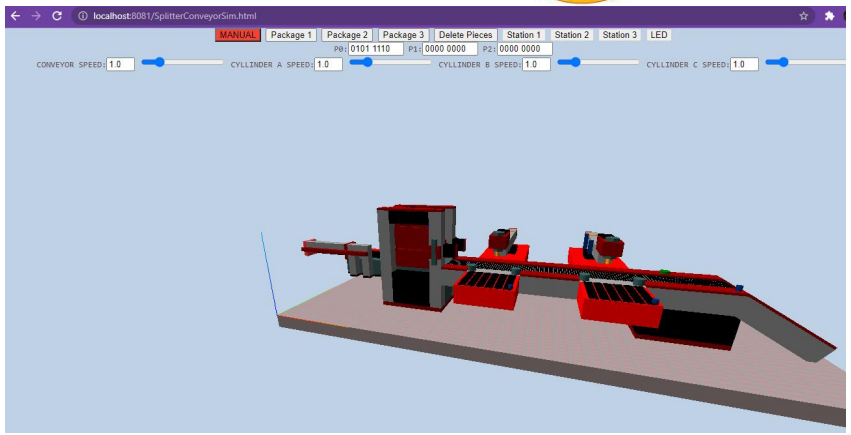
Finally, keep in mind that the system has active high and active low sensors. Active low sensors provide logical value false/0 when active, and true/1 otherwise.

The functional **requirements** to develop the **Splitter Lego Conveyor** are listed in the following table. They are split into basic or low-level requirements, for direct interaction with sensors and actuators, and high-level requirements, for storage management.

Table 1 – Needed requirements.

Req.	Description
Low-Level Requirements:	
FR1.1	Move and stop each **cylinder**
FR1.2	Move and stop moving the **conveyor**
FR1.3	Read **Brick identification** sensors
FR1.4	Detect the **button/switches** states
FR1.5	Turn **Lamp** on and off
FR1.6	Keyboard or UI interaction
High-Level requirements (operation):	
FR2	Using the keyboard, develop robust (semi-automatic) **calibration**. Using keys, the user defines the direction of the movements. During a movement, the actuator must stop when it reaches a position sensor (Cylinders)
FR3	Insert a brick into the system, which gets a brick from the entry queue and puts it in the conveyor
FR4	By pressing "s" the system Starts its working cycle. Pressing "f", the system Finishes its work.
FR5	Deliver **Brick1** for **Dock 1**, **Brick2** for **Dock 2**, and **Brick3** to **DockEnd**
FR6	Detection of **buttons** P1.4 and P1.3 pressed by human operator, at dock 1 and dock 2 to force specific bricks to the corresponding docks
FR7	Flash lamp at P2.7 once when delivering sequence at Dock 1 or Dock 2

- Let's architect our project solution!

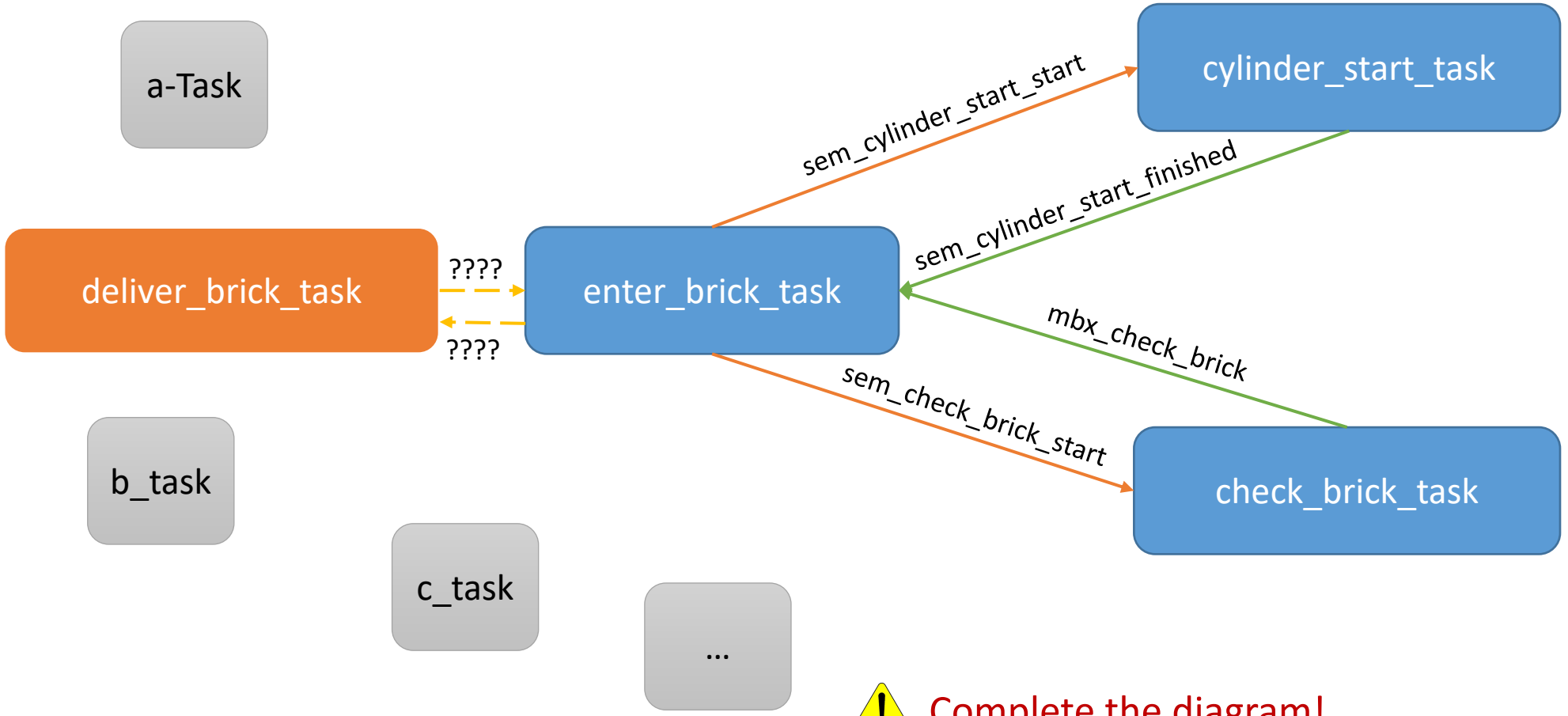


- Need a solution to perform the required functionalities!
  - Menu showing functionalities
  - Request user to insert brick
  - Performing identification task
  - Sending brick to correct dock
  - Etc…

  - Which tasks?
  - Need tasks synchronization?
  - Need tasks communication? (information exchange)

Lab Work 1

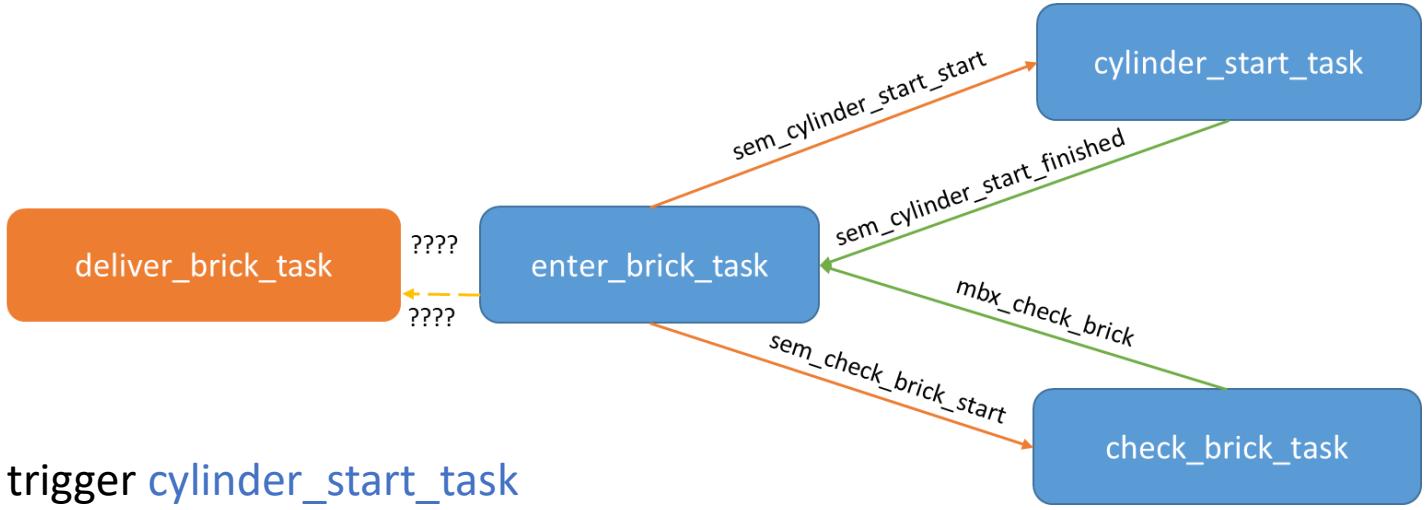
- Let's start by designing a diagram of tasks and inter-task interactions:



Complete the diagram!

Lab Work 1

- Implementing the designed tasks and interactions (the blue ones so far…)



Some suggestions….

- ✓ Creation of semaphore sem_cylinder_start_start to trigger cylinder_start_task
- ✓ Creation of semaphore sem_check_brick_start to trigger check_brick_task
- ✓ Creation of semaphore sem_cylinder_start_finished to trigger enter_brick_task
- ✓ Creation of mailbox mbx_check_brick to communicate with enter_brick_task
- ✓ Creation of enter_brick_task
- ✓ Creation of cylinder_start_task
- ✓ Creation of check_brick_task
- ✓ Check interrupts (example P1.4)

Lab Work 1

- Definition and creation of semaphores and mailboxes

```
//Semaphores and Mailboxes declaration
xSemaphoreHandle sem_cylinder_start_start;
xSemaphoreHandle sem_check_brick_start;
xSemaphoreHandle sem_cylinder_start_finished;

xQueueHandle mbx_check_brick;
```

```
void myDaemonTaskStartupHook(void) {
    inicializarPortos();

    sem_cylinder_start_start = xSemaphoreCreateCounting(10, 0);
    sem_check_brick_start = xSemaphoreCreateCounting(10, 0);
    sem_cylinder_start_finished = xSemaphoreCreateCounting(10, 0);

    mbx_check_brick = xQueueCreate(10, sizeof(int));

}
```

Lab Work 1

- Definition and creation of `cylinder_start_task`, `check_brick_task` and `enter_brick_task`

```
void cylinder_start_task (void* pvParameters) {
   while (TRUE) {
      xQueueSemaphoreTake(sem_cylinder_start_start,
portMAX_DELAY);
      gotoCylinderStart(1);
      xSemaphoreGive(sem_cylinder_start_finished);
      gotoCylinderStart(0); // signals task completion
   }
}
```

```
void check_brick_task(void* pvParameters) {
   int brickType;
   while (TRUE) {
      xQueueSemaphoreTake(sem_check_brick_start,
           portMAX_DELAY);
      brickType=1;   // to be replaced by check brick function later
      xQueueSend(mbx_check_brick, &packageType,
           portMAX_DELAY);
                     // task completion
    }
}
```

send request for cylinder_start_task and check_brick_task which execute gotoCylinderStart and checkBrick on their own.

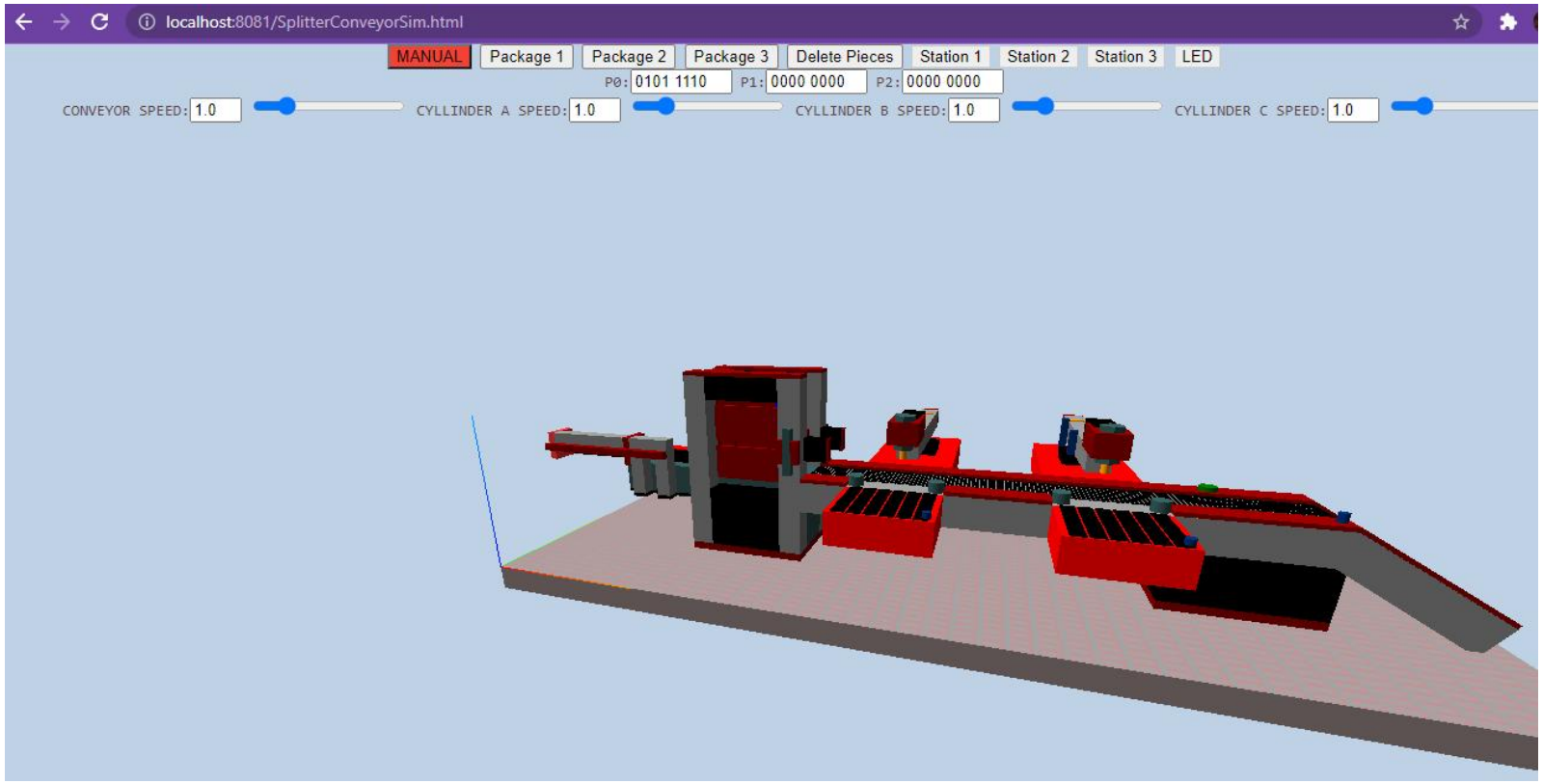
```
void enter_brick_task(void* pvParameters) {
    int brickType;
    while (TRUE) {
        _getch();    // press a key to enter a new package
        xSemaphoreGive(sem_cylinder_start_start);
        xSemaphoreGive(sem_check_brick_start);

        xQueueSemaphoreTake(sem_cylinder_start_finished, portMAX_DELAY);
        xQueueReceive(mbx_check_brick, &brickType, portMAX_DELAY);

        printf("Brick received, Brick Type: %d", brickType);
    }
}
```

// wait for tasks completion

```
void myDaemonTaskStartupHook(void) {
    inicializarPortos();
    sem_cylinder_start_start = xSemaphoreCreateCounting(10, 0);
    sem_check_brick_start = xSemaphoreCreateCounting(10, 0);
    sem_cylinder_start_finished = xSemaphoreCreateCounting(10, 0);
    mbx_check_brick = xQueueCreate(10, sizeof(int));
    xTaskCreate(cylinder_start_task, "Cylinder Start Task", 100, NULL, 0, NULL);
    xTaskCreate(check_brick_task, "Check brick Task", 100, NULL, 0, NULL);
    xTaskCreate(enter_brick_task, "Enter brick Task", 100, NULL, 0, NULL);
}
```

Lab Work 1

- Run the project…



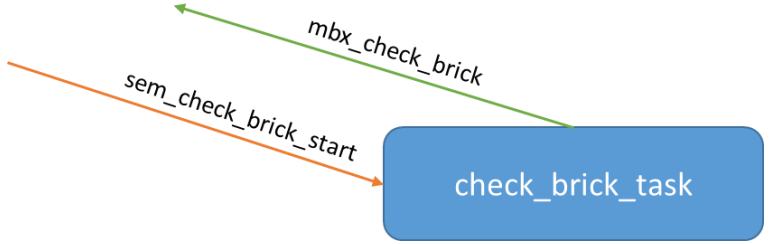
Passing Parameters in Task Creation

- As known, it is not a **good practice** to use global variables, especially in a real time context!!!

```
//Semaphores and Mailboxes declaration
xSemaphoreHandle sem_cy                rt_start;
xSemaphoreHandle sem_ch                e_start;
xSemaphoreHandle sem_cy                rt_finished;

xQueueHandle mbx_check_
```

- Let's consider the creation of check_brick_task. This task uses a semaphore (check_brick_start) and a mailbox (mbx_check_brick).
- It is then necessary to pass both task mechanisms in the **creation of the task**.
- For that, we suggest to create a structure with both mechanism handlers:

```
typedef struct taskCheckBrickParams {
    xQueueHandle mbx_check_brick;
    xSemaphoreHandle sem_check_brick_start;
} TaskCheckBrickParam;
```

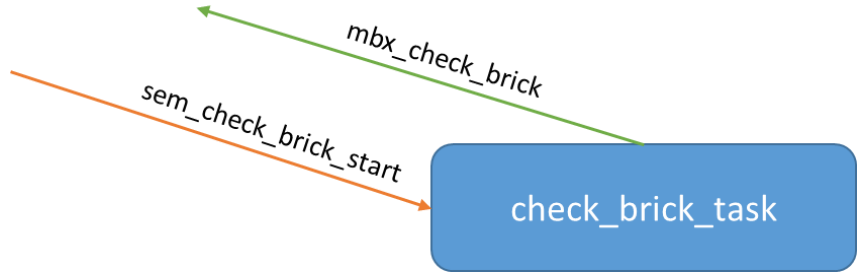


The way you organize your code/structures is up to you! This is just an example.

Passing Parameters in Task Creation

- In `myDaemonTaskStartupHook` function:
  1 – create the corresponding mechanism handlers;
  2 – dynamically allocate memory to the structure (using the `pvPortMalloc` freeRTOS functionality) and assign value to its attributes;
  3 – pass as argument the dynamically created pointer to the structure.

Attention! It is not recomended to pass the address of a stack variable!




```
void myDaemonTaskStartupHook(void) {

    inicializarPortos();

    xQueueHandle mbx_check_brick = xQueueCreate(10, sizeof(int));
    xSemaphoreHandle sem_check_brick_start = xSemaphoreCreateCounting(10, 0);
    /* ... others ... */

    TaskCheckPackage *my_taskcheckbrick_Params = (TaskCheckBrickParams*)pvPortMalloc(sizeof(TaskCheckBrickParams));
    my_taskcheckbrick_Params->mbx_check_brick = mbx_check_brick;
    my_taskcheckbrick_Params->sem_check_brick_start = sem_check_brick_start;
    /*... others ...*/

    xTaskCreate(check_brick_task, "Check brick Task", 100, my_taskcheckbrick_Params, 0, NULL);
    /* ... others ...*/
}
```

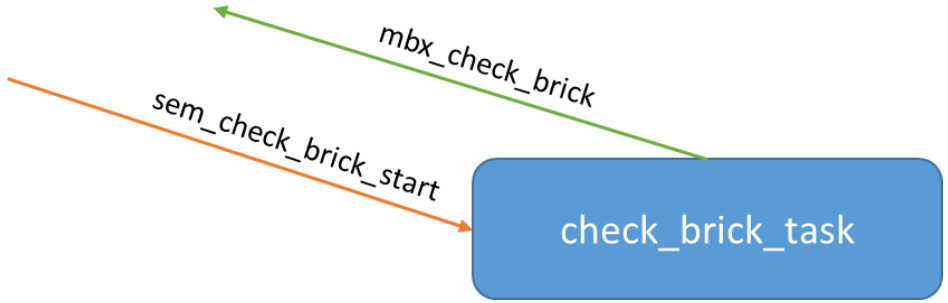
1

2

3

Passing Parameters in Task Creation

- In `check_brick_task`:
  - receive the pointer to the structure and the corresponding fields



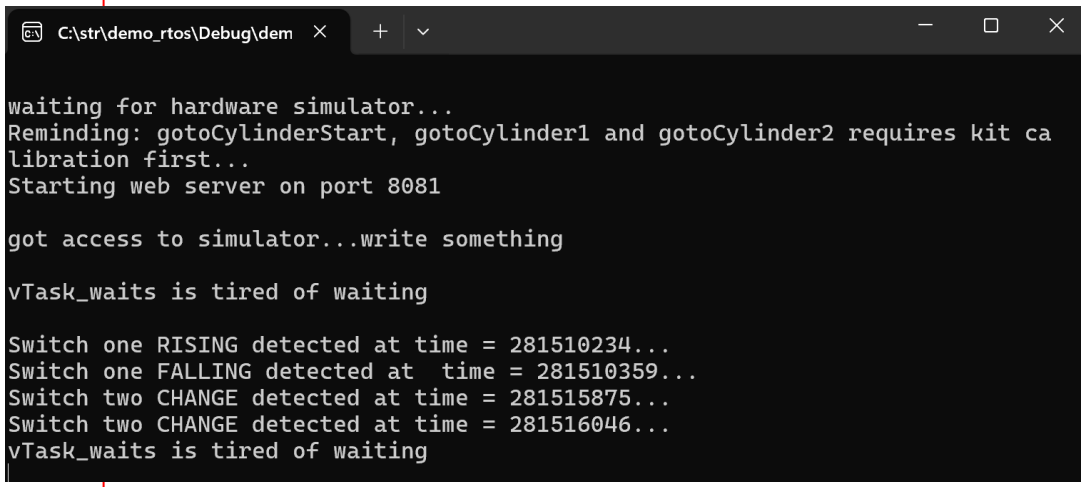

```
void check_brick_task(void* pvParameters) {
    int z;

    TaskCheckBrickParams* my_params = (TaskCheckBrickParams*)pvParameters;
    xQueueHandle mbx_check_brick = my_params->mbx_check_brick;
    xSemaphoreHandle sem_check_brick_start = my_params-> sem_check_brick_start;

    int brickType;
    while (TRUE) {
      xQueueSemaphoreTake(sem_check_brick_start,        portMAX_DELAY);
      brickType=1;
      xQueueSend(mbx_check_brick, &brickType,           portMAX_DELAY);
    }
}
```

Interrupts

REVIEW

- Using Interrupt Service Routines (ISR) (on port bit values)

```
void switch1_rising_isr(ULONGLONG lastTime) {
  // GetTickCount64() current time in miliseconds
  // since the system has started...
  ULONGLONG  time = GetTickCount64();
  printf("\nSwitch one RISING detected at time = %llu...", time);
}

void switch1_falling_isr(ULONGLONG lastTime) {
  ULONGLONG  time = GetTickCount64();
  printf("\nSwitch one FALLING detected at time = %llu...", time);
}

void switch2_change_isr(ULONGLONG lastTime) {
  ULONGLONG  time = GetTickCount64();
  printf("\nSwitch two CHANGE detected at time = %llu...", time);
}
```




```
void myDaemonTaskStartupHook(void) {
   // …code already here
   attachInterrupt(1, 4, switch1_rising_isr, RISING);
   attachInterrupt(1, 4, switch1_falling_isr, FALLING);
   attachInterrupt(1, 3, switch2_change_isr, CHANGE);
}
```

INTERRUPTS [REVIEW]

- Using Interrupt Service Routines (ISR) on port values

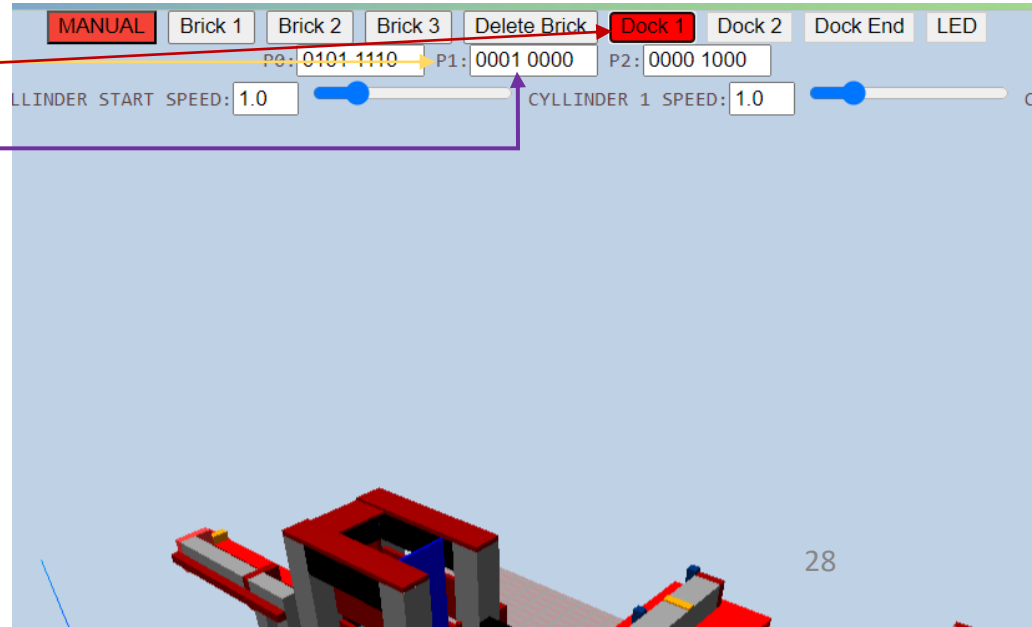
freeRTOS/include -> interrupt.h:

```
void attachInterrupt(int port, int bit, void(*ISR)(ULONGLONG lastTime), int Mode);
```

Example:

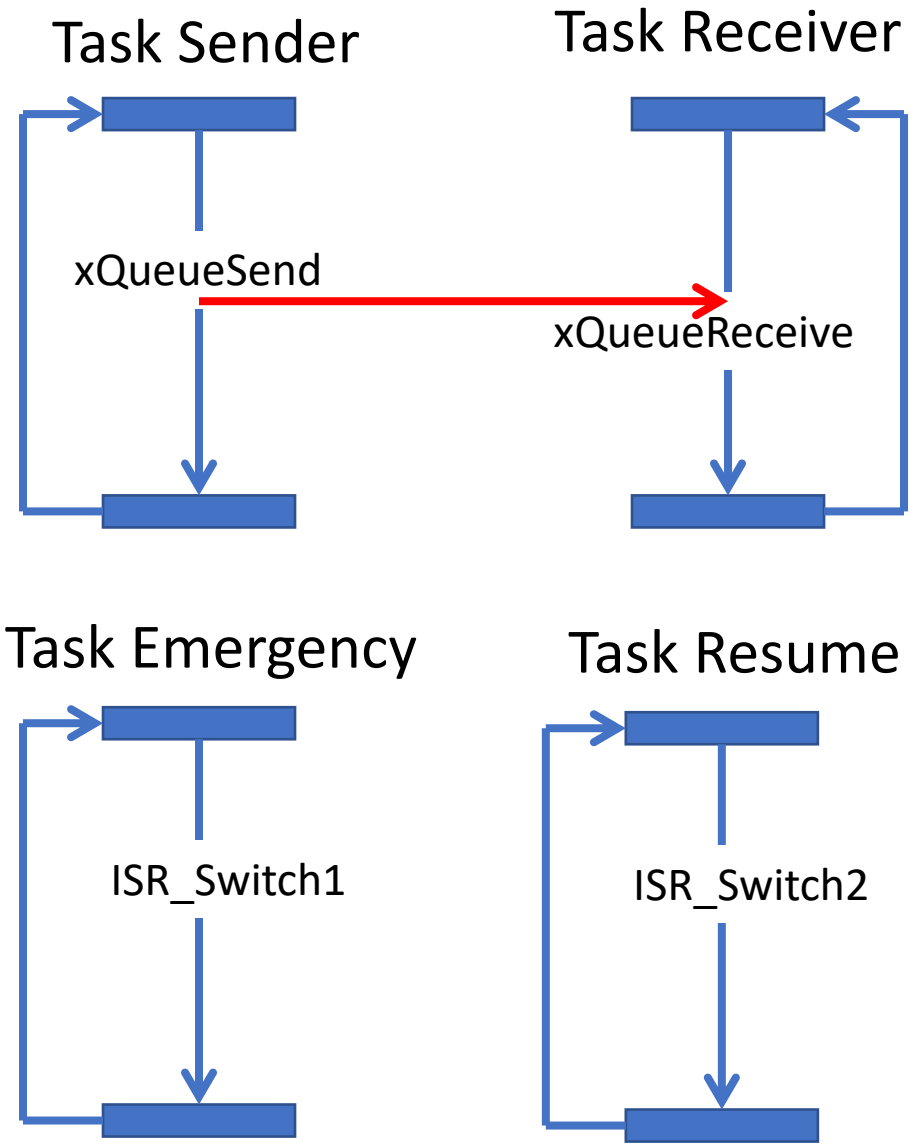
switch1 -> p1.4 (active high)

```
void myDaemonTaskStartupHook(void) {
   // …code already here
   attachInterrupt(1, 4, switch1_rising_isr, RISING);
   attachInterrupt(1, 4, switch1_falling_isr, FALLING);
   attachInterrupt(1, 3, switch2_change_isr, CHANGE);
}
```



Mode		
	LOW	Triggers interrupt whenever the pin is LOW
	HIGH	Triggers interrupt whenever the pin is HIGH
	FALLING	Triggers interrupt when the pin goes from HIGH to LOW
	RISING	Triggers interrupt when the pin goes from LOW to HIGH
	CHANGE	Triggers interrupt whenever the pin changes value, from HIGH to LOW or LOW to HIGH

INTERRUPTS & TASK CONTROL



- Two parallel tasks exchanging data through a MAILBOX
- One emergency task triggered by an interrupt -> switch (suspends task sender)
- One resume task triggered by an interrupt -> switch (resumes task sender)

INTERRUPTS & TASK CONTROL

- Let's update project **demo_rtos.sln** (in C:\str\demo_rtos) to try this example:

```
// tasks handler declaration
xTaskHandle emergencyTask;
xTaskHandle resumeTask;
xTaskHandle taskSender;
```

Emergency and Resume Tasks Creation & Implementation

```
void vTaskEmergency(void* pvParameters){
    // The task being suspended and resumed.
    for (;; ){
       // The task suspends itself.
       vTaskSuspend(NULL);
       // The task is now suspended, so will
       not reach here until the ISR resumes it.
       printf("\n **** EMERGENCY task\n");
       // The task suspends task sender.
       vTaskSuspend(taskSender);
    }
}
```

```
void vTaskResume(void* pvParameters){
    // The task being suspended and resumed.
    for (;; ){
       // The task suspends itself.
       vTaskSuspend(NULL);
       // The task is now suspended, so will
       not reach here until the ISR resumes it.
       printf("\n **** RESUME task\n");
       // The task suspends task sender.
       vTaskResume(taskSender);
    }
```

The handler of the **taskSender** needs to be known!

```
void myDaemonTaskStartupHook(void) {
    inicializarPortos();
    // *** some code here
    xTaskCreate(vTaskEmergency, "vTaskEmergency ", 100, NULL, 0, &emergencyTask);
    xTaskCreate(vTaskResume, "vTaskResume ", 100, NULL, 0, &resumeTask);
}
```

The handler of the task will be needed by the ISR!

INTERRUPTS & TASK CONTROL

ISR Routines Implementation

```
void switch1_rising_isr(ULONGLONG lastTime) {
    // GetTickCount64() current time in miliseconds
    // since the system has started...
    ULONGLONG  time = GetTickCount64();
    printf("\nSwitch one RISING detected at time = %llu...\n", time);

    BaseType_t xYieldRequired;
    // Resume the suspended task.
    xYieldRequired = xTaskResumeFromISR(emergencyTask);
}
```

```
void switch2_rising_isr(ULONGLONG lastTime) {
    ULONGLONG  time = GetTickCount64();
    printf("\nSwitch two RISING detected at time = %llu...", time);

    BaseType_t xYieldRequired;
    // Resume the suspended task.
    xYieldRequired = xTaskResumeFromISR(resumeTask);
}
```

INTERRUPTS & TASK CONTROL

In myDaemonTaskStartupHook…

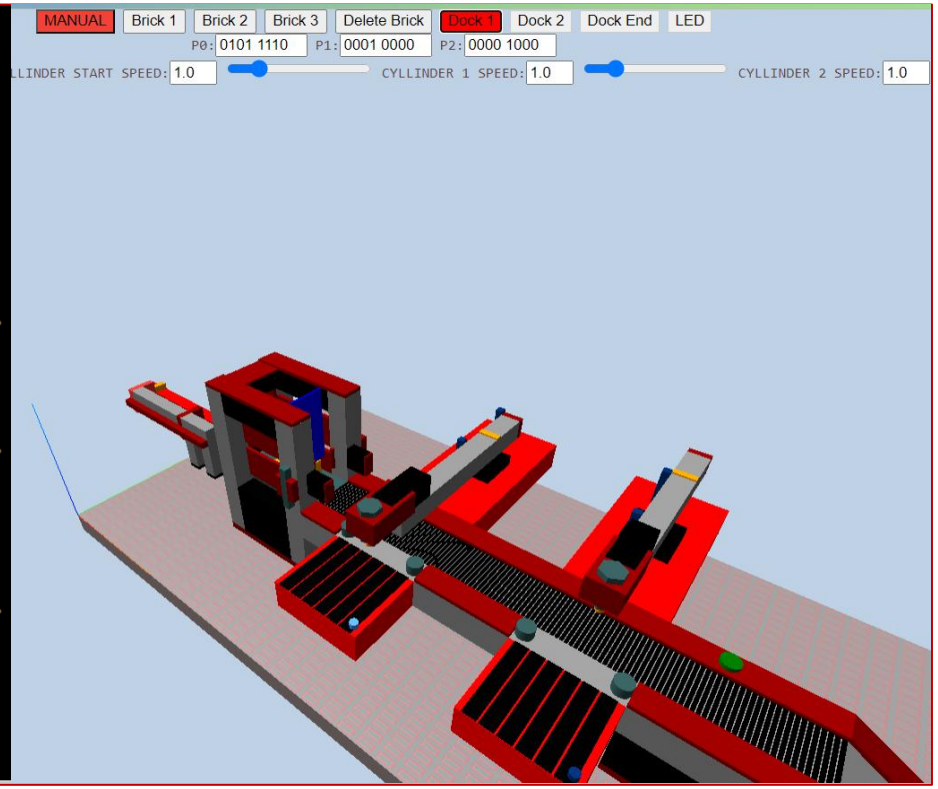
```
void myDaemonTaskStartupHook(void) {
    inicializarPortos();

    // *** some code here

    xQueueHandle xQueue = xQueueCreate(10, sizeof(double));
    xTaskCreate(vTaskCode_MbxReceiver, "vTaskCode_MbxReceiver ", 100, xQueue, 0, &taskReceiver);
    xTaskCreate(vTaskCode_MbxSender, "vTaskCode_MbxSender ", 100, xQueue, 0, &taskSender);

    xTaskCreate(vTaskEmergency, "vTaskEmergency ", 100, NULL, 0, &emergencyTask);
    xTaskCreate(vTaskResume, "vTaskResume ", 100, NULL, 0, &resumeTask);

    attachInterrupt(1, 4, switch1_rising_isr, RISING);
    attachInterrupt(1, 3, switch2_rising_isr, RISING);

}
```

INTERRUPTS & TASK CONTROL

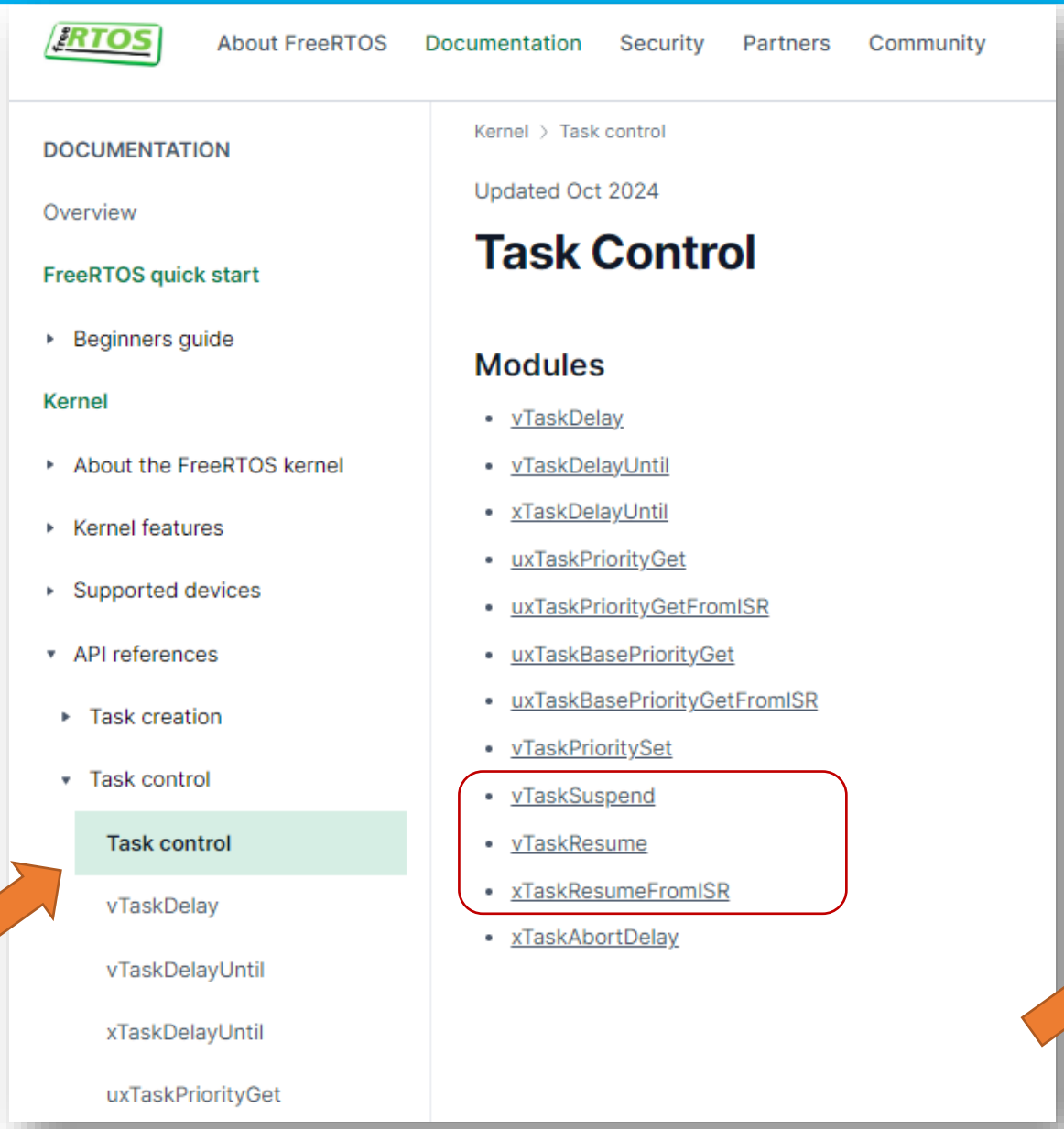
```
C:\str\demo_rtos\Debug\demo_rtos.exe

waiting for hardware simulator...

Starting web server on port 8081

got access to simulator...
received_val =1.000
received_val =2.000
received_val =3.000
received_val =4.000
received_val =5.000
received_val =6.000
received_val =7.000
received_val =8.000
received_val =9.000
Switch one RISING detected at time = 90579531...

 **** EMERGENCY task
```

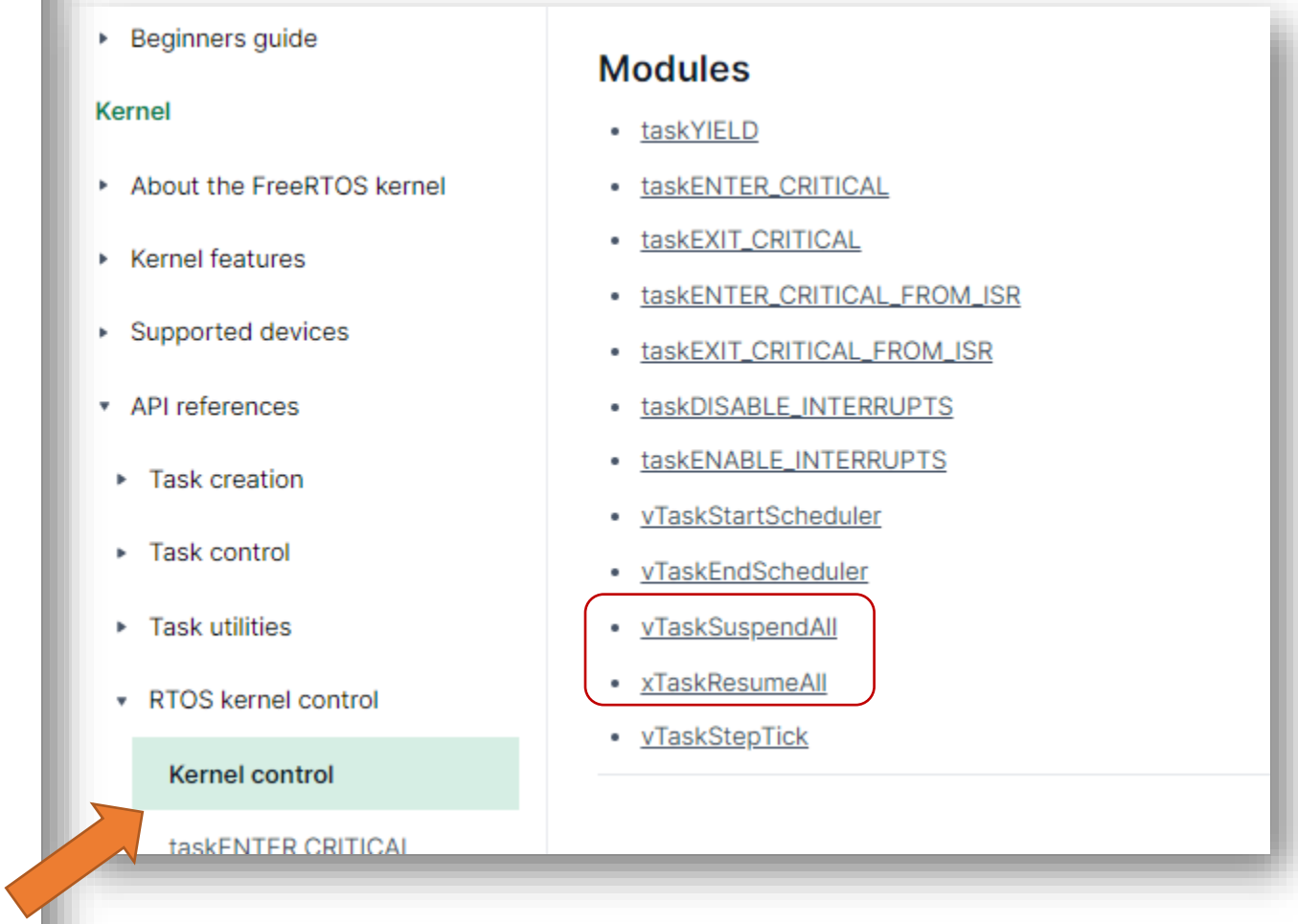
Running the project…

```
eceived_val =1.000
eceived_val =2.000
eceived_val =3.000
eceived_val =4.000
eceived_val =5.000
eceived_val =6.000
eceived_val =7.000
eceived_val =8.000
eceived_val =9.000
witch one RISING detected at time = 90579531...

 **** EMERGENCY task

witch two CHANGE detected at time = 90694171...
 **** RESUME task

eceived_val =10.000
eceived_val =11.000
witch two CHANGE detected at time = 90699062...
 **** RESUME task

eceived_val =12.000
```



freeRTOS API – TASK CONTROL





AWESOME! LET'S USE TASKS AND THEIR MECHANISMS TO PLAY WITH YOUR LAB 1 SYSTEM IN REAL-TIME SYSTEMS!

- Review Labwork 1 Requirements
- Design a Tasks and Interaction Diagram
- Brainstorm Ideas and Solutions
- Test FreeRTOS Demos
- Apply concepts to Labwork 1

RELAX, EXPLORE, HAVE FUN, AND ENJOY THE RIDE!

KEEP UP THE GOOD WORK!